

ECO 529: Macro-Finance: Tools, Models, and Ideas

Lecture 7: Solving Krusell-Smith: Part 2: NN Solution to Master Equation

Jonathan Payne

Goals

- ▶ Focus on solving Krusell-Smith (2008), which has become a “test” model.
- ▶ Overview of solution techniques:
 1. Perturbation,
 2. Statistical approximation to the law of motion for key moments of the distribution, and
 3. Projection.
- ▶ This lecture: focus on different proposals for doing projection using neural networks.
- ▶ Focus on “*Global Solutions to Master Equations for Continuous Time Heterogeneous Agent Macroeconomic Models.*” by Gu, Lauriere, Payne, and Merkel.

Table of Contents

Simple Macroeconomic Model (Krusell-Smith (1998))

Solution Algorithm

Testing the Algorithm

Additional Models

Related Literature

Recall: “Master Equation” Representation of RE Equilibrium

- ▶ “Master equation” substitutes KFE, market clearing, & belief consistency into HJBE.
- ▶ Equilibrium value function $V(a, l, z, g)$ characterized by one PDE:
(With the change of variable that $l = n$.)

$$\begin{aligned} 0 = & -\rho V(a, l, z, g) + u(c^*(a, l, z, g)) + \mathbf{1}_{a_t \leq \underline{a}} \psi(a_t) \\ & + \partial_a V(a, l, z, g)(w(z, g)l + r(z, g)a - c^*(a, l, z, g)) \\ & + \lambda(x) \left(V(a, \check{l}, z, g) - V(a, l, z, g) \right) + \partial_z V(a, l, z, g) \mu^z(z) + 0.5 (\sigma^z)^2 \partial_{zz} V(a, l, z, g) \\ & + \sum_{j \leq 2} \int_{\underline{a}}^{\infty} \frac{\partial V}{\partial g}(a, l_j, z, g(y, l_j)) \mu^g(c^*(y, n_j, z, g), y, l_j, z, g) dy =: \mathcal{L}V \end{aligned}$$

where the optimal control c^* is characterised by:

$$u'(c^*(a, l, z, g)) = \partial_a V(a, l, z, g).$$

We find it easier to work with $\partial_a V(a, l, z, g)$ rather than $V(a, l, z, g)$. Why?

Master Equation for $\partial_a V(a, l, z, g)$

- Define $W(a, l, z, g) := \partial_a V(a, l, z, g)$.
- “Master equation” for $W(a, l, z, g)$: (using the envelope theorem.)

$$\begin{aligned}
 0 = & (r(z, g) - \rho)W(a, l, z, g) + \mathbf{1}_{a_t \leq \underline{a}} \psi(a_t) \\
 & + \underbrace{\partial_a W(a, l, z, g)(w(z, g)l + r(z, g)a - c^*(a, l, z, g)) + \lambda(l) \left(V(a, \check{l}, z, g) - W(a, l, z, g) \right)}_{=:\mathcal{L}_x W(a, l, z, g)} \\
 & + \underbrace{\partial_z W(a, l, z, g) \mu^z(z) + 0.5 (\sigma^z)^2 \partial_{zz} W(a, l, z, g)}_{=:\mathcal{L}_z W(a, l, z, g)} \\
 & + \underbrace{\sum_{j \leq 2} \int_{\underline{a}}^{\infty} \frac{\partial W}{\partial g}(a, l_j, z, g(y, l_j)) \mu^g(c^*(y, n_j, z, g), y, l_j, z, g) dy}_{=:\mathcal{L}_g W(a, l, z, g)} =: \mathcal{L}W
 \end{aligned}$$

where the optimal control c^* is characterised by: $u'(c^*(a, l, z, g)) = W(a, l, z, g)$.

Table of Contents

Simple Macroeconomic Model (Krusell-Smith (1998))

Solution Algorithm

Testing the Algorithm

Additional Models

Numerical Approximation Outline

- ▶ Goal: characterize approximate solution to Master equation numerically.
- ▶ Problem: Master equation contains an infinite dimensional derivative.
- ▶ Solution: three main ingredients:
 1. High but finite dimensional approximation to distribution and Master equation:
 - (i). Replace continuum of agents by a **finite population** of agents, or
 - (ii). **Discretize** the wealth variable, or
 - (ii). **Project** distribution onto a finite dimensional set of basis functions $b_i(x)$ (e.g. eigenfunctions, Chebyshev polynomials, ...).
 2. Parameterize W by neural network, and
 3. Train the parameters to minimize the (approximate) master equation residual.

Ingredient 1: Finite Dimensional “Distribution” Approximation

	Finite Population	Discrete State	Projection
Dist. approx. params $\hat{\varphi}_t$	Agent states $\hat{\varphi}_t = \{(a_t^i, l_t^i)\}_{i \leq N}$	Masses on grid	Basis coefficients
Dist. approx. $\hat{g}_t$	$\frac{1}{N} \sum_{i=1}^N \delta_{\hat{\varphi}_t^i}$	$\sum_{i=1}^N \hat{\varphi}_{i,t} \delta_{(a_i, l_i)}$	$b_0(a, l) + \sum_{i=1}^N \hat{\varphi}_{i,t} b_i(a, l)$
KFE approx. ($\mu_{\hat{\varphi}}$)	Evolution of other agents' states	Evolution of mass between grid points (e.g. finite diff.)	Evolution of projec- tion coefficients (least squares)
Dimension (N)	≈ 40	≈ 200	≈ 5
$K_t =$	$\frac{1}{N} \sum_i^N a_t^i$	$\sum_j \sum_{i=1}^N a^i \hat{\varphi}_{i,t}$	$\sum_j \int a \sum_i \hat{\varphi}_{i,t} b_i(a; l_j) da$

Ingredient 1: Comparison of Approximation Methods

- ▶ Dimensionality:
 - ▶ **Finite Population**: sufficiently large to average out idiosyncratic noise
 - ▶ **Discrete State**: sufficiently fine to approximate derivatives in KFE well
 - ▶ **Projection**: potentially low for efficient basis choice
- ▶ Required customization:
 - ▶ **Finite Population**: choose number of agents (N)
 - ▶ **Discrete State**: choose grid and method for approximating derivatives in KFE on grid
 - ▶ **Projection**: choose basis and which aspects of KFE to approximate [see next slide]

Projection potentially lower-dimensional but requires more intricate choices

Ingredient 1a: Finite Population: Master Equation

$$0 = \mathcal{L}\hat{W}((a^i, l^i), z, \hat{\varphi}) = (r(z, \hat{\varphi}^{-i}) - \rho)\hat{W}((a^i, l^i), z, \hat{\varphi}) + \mathbf{1}_{a \leq a_{lb}} \partial_a \psi(a^i) \\ + (\mathcal{L}_x + \mathcal{L}_z + \hat{\mathcal{L}}_g)\hat{W}((a^i, l^i), z, \hat{\varphi})$$

where the operators become:

$$\mathcal{L}_x \hat{W}((a^i, l^i), z, \hat{\varphi}) = s((a^i, l^i), \hat{c}^*((a^i, l^i), z, \hat{\varphi}), r(z, \hat{\varphi}^{-i}), w(z, \hat{\varphi}^{-i})) \partial_{a^i} \hat{W}((a^i, l^i), z, \hat{\varphi}) \\ + \lambda(l^i) \left(\hat{W}((a^i, \check{l}^i), z, \hat{\varphi}) - \hat{W}((a^i, l^i), z, \hat{\varphi}) \right)$$

$$\mathcal{L}_z \hat{W}((a^i, l^i), z, \hat{\varphi}) = \partial_z \hat{W}((a^i, l^i), z, \hat{\varphi}) \eta(\bar{z} - z) + \frac{1}{2} \sigma^2 \partial_{zz} \hat{W}((a^i, l^i), z, \hat{\varphi})$$

$$\hat{\mathcal{L}}_g \hat{W}((a^i, l^i), z, \hat{\varphi}) = \sum_{j \neq i} s((a^j, l^j), \hat{c}^*((a^j, l^j), z, \hat{\varphi}), r(z, \hat{\varphi}^{-j}), w(z, \hat{\varphi}^{-j})) \partial_{a^j} \hat{W}((a^i, l^i), z, \hat{\varphi}) \\ + \lambda(l^j) \left(\hat{W}((a^i, l^i), \{(a^j, \check{l}^j), z, \hat{\varphi}^{-j}\}) - \hat{W}((a^i, l^i), z, \hat{\varphi}) \right)$$

Ingredient 1a: Finite Population: Why Are 40 Agents Sufficient?

Simulation-based finite-agent methods in discrete time often need a large number of agents to be accurate. Why not here?

1. We sample individual states (x) and distribution states ($\hat{\varphi}$) separately when generating training sets
 - ▶ Without separation: need large N to have sufficiently many x in high-curvature regions (e.g., close to borrowing constraint) to learn value function well.
 - ▶ With separation: can independently control where we learn V in the x -dimension and how we represent g .
 - ▶ Ultimately, 40 agents is sufficient for approximating K because LLN kicks in quickly.
2. We can eliminate/reduce finite agent noise
 - ▶ In $\hat{\varphi}$ -drift during training, can eliminate it analytically.
 - ▶ When simulating trained model: approximate KFE operator with many simulations instead of simulating finite population economy.

Ingredient 1: Finite Population: Simulation

- 1: Start from an initial g_0 equal to the ergodic distribution.
 - 2: **for** $n = 0, \dots, N_T - 1$ **do**
 - 3: Sample ΔB_t^0 from normal distribution $N(0, \Delta t)$, construct TFP shock path by:
 $z_{t+\Delta t} = z_t + \eta(\bar{z} - z_t) + \sigma \Delta B_t^0$.
 - 4: **for** $k = 1, \dots, N_{sim}$ **do**
 - 5: Draw states for N agents $\{\varphi_i^k : i = 1, \dots, N\}$ from density g_t at $t = n\Delta t$.
 - 6: At state (z_t, φ_t^k) , get eqm return $r_t^k = \hat{r}(z_t, \varphi_t^k)$ and wage $w_t^k = \hat{w}(z_t, \varphi_t^k)$.
 - 7: **end for**
 - 8: Calculate the average $\bar{r}_t = \frac{1}{N_{sim}} \sum_{k=1}^{N_{sim}} r_t^k$ and $\bar{w}_t = \frac{1}{N_{sim}} \sum_{k=1}^{N_{sim}} w_t^k$
 - 9: At each grid point $a_m \in \underline{a}$, calculate consumption $\hat{c}((a_m, l), z_t, \bar{r}_t, \bar{w}_t)$, $\forall l$.
 - 10: Construct the transition matrix $\mathcal{A}_t$ using finite difference on the grid $\underline{a}$
 - 11: Update g_t by implicit method: $g_{t+\Delta t} = (I - \bar{\mathcal{A}}_t^\top \Delta t)^{-1} g_t$
 - 12: **end for**
-

Ingredient 1b: Discrete State Space: Master Equation

$$0 = \mathcal{L}\hat{W}((a, l), z, \hat{\varphi}) = (r(z, \hat{\varphi}) - \rho)\hat{W}((a, l), z, \hat{\varphi}) + \mathbf{1}_{a \leq a_{lb}} \partial_a \psi(a) \quad (1)$$

$$+ (\mathcal{L}_x + \mathcal{L}_z + \hat{\mathcal{L}}_g)\hat{W}((a, l), z, \hat{\varphi}) \quad (2)$$

where the operators become:

$$\mathcal{L}_x \hat{W}((a, l), z, \hat{\varphi}) = s((a, l), \hat{c}^*((a, l), z, \hat{\varphi}), r(z, \hat{\varphi}), w(z, \hat{\varphi})) \partial_a \hat{W}((a, l), z, \hat{\varphi}) \quad (3)$$

$$+ \lambda(l) \left(\hat{W}((a, \check{l}), z, \hat{\varphi}) - \hat{W}((a, l), z, \hat{\varphi}) \right) \quad (4)$$

$$\mathcal{L}_z \hat{W}((a, l), z, \hat{\varphi}) = \partial_z \hat{W}((a, l), z, \hat{\varphi}) \eta(\bar{z} - z) + \frac{1}{2} \sigma^2 \partial_{zz} \hat{W}((a, l), z, \hat{\varphi}) \quad (5)$$

$$\hat{\mathcal{L}}_g \hat{W}((a, l), z, \hat{\varphi}) = \sum_{j=1,2} \sum_{m=1}^N \mu_{\hat{\varphi}, m, j}(z, \hat{\varphi}) \partial_{\hat{\varphi}_{m, j}} \hat{W}((a, l), z, \hat{\varphi}) \quad (6)$$

where $\partial_{\hat{\varphi}_{m, j}} \hat{W}$ denotes the partial derivative of $\hat{W}$ with respect to the coordinate (m, j) of $\hat{\varphi}$ and where $\mu_{\hat{\varphi}, m, j}(z, \hat{\varphi})$ is the finite difference approximation.

Ingredient 1c: Additional Details on Projection Method

Projection method requires two additional choices:

1. Law of motion of coefficients $\hat{\varphi}_t$

- ▶ Goal: find drifts $\mu_{\hat{\varphi},i,t}$, $i = 1, \dots, N$ such that for all $x = (a, l)$

$$\underbrace{\sum_{i=1}^N \mu_{\hat{\varphi},i,t} b_i(a, l)}_{=\partial_t \hat{g}_t(a, l)} \approx \mu_g \left(a, l, z_t, b_0 + \underbrace{\sum_{i=1}^N \hat{\varphi}_{i,t} b_i}_{=\hat{g}_t} \right).$$

- ▶ Problem: these are infinite equations, but we have only N degrees of freedom.
- ▶ Our approach: choose $M \geq N$ “test functions” $\phi(a, l)$ and approximate law of motion of “statistics” $\sum_j \int \phi(a, l_j) \hat{g}_t(a, l_j) da$ in least squares sense.
(One “informative” statistic for KS is $\phi(a, l) = a$ to approximate dK_t well)

2. Choice of Basis (our approach)

Ingredient 1c: Additional Details on Projection Method

Projection method requires two additional choices:

1. Law of motion of coefficients $\hat{\varphi}_t$
2. Choice of Basis (our approach)
 - ▶ Eigenfunctions of steady-state KFE operator $\bar{\mathcal{L}}^{KF}$ corresponding to largest eigenvalues.
 - ▶ If KFE was $dg_t = \bar{\mathcal{L}}^{KF} g_t dt$, these would be the most persistent components of g_t .
 - ▶ Actual KFE is $dg_t = \mathcal{L}_t^{KF} g_t dt$; they are still (very) persistent components if $\mathcal{L}_t^{KF} \approx \bar{\mathcal{L}}^{KF}$.
 - ▶ Why (most) persistent components?
 - ▶ g_t only enters value function because it helps forecast future prices q_{t+T} .
 - ▶ most persistent components carry most information (for large T).

Ingredient 1c: Projection: Master Equation

$$0 = \mathcal{L}\hat{W}((a, l), z, \hat{\varphi}) = (r(z, \hat{\varphi}) - \rho)\hat{W}((a, l), z, \hat{\varphi}) + \mathbf{1}_{a \leq a_{lb}} \partial_a \psi(a) \quad (7)$$

$$+ (\mathcal{L}_x + \mathcal{L}_z + \hat{\mathcal{L}}_g)\hat{W}((a, l), z, \hat{\varphi}) \quad (8)$$

where the operators become:

$$\mathcal{L}_x \hat{W}((a, l), z, \hat{\varphi}) = s((a, l), \hat{c}^*((a, l), z, \hat{\varphi}), r(z, \hat{\varphi}), w(z, \hat{\varphi})) \partial_a \hat{W}((a, l), z, \hat{\varphi}) \quad (9)$$

$$+ \lambda(l) \left(\hat{W}((a, \check{l}), z, \hat{\varphi}) - \hat{W}((a, l), z, \hat{\varphi}) \right) \quad (10)$$

$$\mathcal{L}_z \hat{W}((a, l), z, \hat{\varphi}) = \partial_z \hat{W}((a, l), z, \hat{\varphi}) \eta(\bar{z} - z) + \frac{1}{2} \sigma^2 \partial_{zz} \hat{W}((a, l), z, \hat{\varphi}) \quad (11)$$

$$\hat{\mathcal{L}}_g \hat{W}((a, l), z, \hat{\varphi}) = \sum_{n=1}^5 \mu_{\hat{\varphi}, n}(z, \hat{\varphi}) \partial_{\hat{\varphi}_n} \hat{W}((a, l), z, \hat{\varphi}) \quad (12)$$

Ingredient 2: Approximate V by Neural Network

- ▶ Let $\omega = (a, l, z, \hat{\phi})$. We approximate value $V(\omega)$ by neural network with form:

$$\mathbf{h}^{(1)} = \phi^{(1)}(W^{(1)}\omega + \mathbf{b}^{(1)}) \quad \dots \text{Hidden layer 1}$$

$$\mathbf{h}^{(2)} = \phi^{(2)}(W^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}) \quad \dots \text{Hidden layer 2}$$

$$\vdots$$

$$\mathbf{h}^{(H)} = \phi^{(H)}(W^{(H)}\mathbf{h}^{(H-1)} + \mathbf{b}^{(H)}) \quad \dots \text{Hidden layer H}$$

$$V = \sigma(W^{(H+1)}\mathbf{h}^{(H)} + \mathbf{b}^{(H+1)}) \quad \dots \text{Output}$$

- ▶ Terminology: a fully connected feed forward NN (finite agent params in blue):
 - ▶ H : is the number of *hidden layers*, ($H = 5$)
 - ▶ Length of vector $\mathbf{h}^{(i)}$: number of *neurons* in hidden layer i , ($Length = 64$)
 - ▶ $\phi^{(i)}$: is the *activation function* for hidden layer i , ($\phi^i = \tanh$)
 - ▶ σ : is the *activation function* for the final layer, (soft-plus)
 - ▶ $\Theta = (W^1, \dots, W^{(H+1)}, b^{(1)}, \dots, b^{(H+1)})$ are the *parameters*,
- ▶ Discrete state, projection more complicated NN [Sirignano and Spiliopoulos, 2018].

Ingredient 3: Algorithm (“EMINN”)

- 1: Initialize the neural networks approximation for $\hat{V}$ with parameters Θ .
- 2: **while** Loss > tolerance **do**
- 3: Sample M new training points: $\mathbf{S} = (a_m, l_m, z_m, \hat{\varphi}_m)_{m=1}^M$.
- 4: Calculate the **weighted average error** across sample points, given current Θ :

$$\mathcal{E}(\mathbf{S}; \Theta) = \kappa^e \frac{1}{M} \sum_{m \leq M} |\hat{\mathcal{L}}(a_m, l_m, z_m, \hat{\varphi}_m; \Theta)|^2 + \kappa^s \mathcal{E}^s(\mathbf{S}; \Theta), \quad \text{where}$$

- ▶ $\hat{\mathcal{L}}(a_m, l_m, z_m, \hat{\varphi}_m; \Theta)$ is **error in Master equation** $\hat{\mathcal{L}}$ at m -th training point (derivates in $\hat{\mathcal{L}}$ are calculated using automatic differentiation.)
 - ▶ $\mathcal{E}^s(\mathbf{S}; \Theta)$ is **penalty for “wrong” shape** (e.g. penalty for non-concavity of V)
- 5: Update parameters Θ by **stochastic gradient descent**: $\Theta^{new} = \Theta^{old} - \alpha D_{\Theta} \mathcal{E}(\mathbf{S}; \Theta)$
 - 6: **end while**
-

The Macroeconomic Deep Learning Conundrum

- ▶ Algorithm is straightforward to describe and code
- ▶ ...but hard to implement successfully!
- ▶ I will discuss some features that we have found helpful.

Sampling Approaches

- ▶ Sampling (a, l, z) : draw from uniform distribution, then add draws where error high (“active sampling”).
- ▶ Sampling the parameters in the distribution approximation $(\hat{\varphi}_i)_{i \leq N}$:
 - ▶ *Moment sampling*:
 1. Draw samples for selected moments of the distribution (that are important for $Q(z, \hat{g})$).
 2. Sample $\hat{\varphi}$ from a distribution that satisfies the moments drawn in the first step.
 - ▶ *Mixed steady state sampling*:
 1. Solve for the steady state for a collection of fixed aggregate states z .
 2. Draw random mixtures of this collection of steady state distributions.
 - ▶ *Ergodic sampling*:
 1. Simulate economy using current value function approximation.
 2. Use simulated distributions as training points.

Need to choose economically relevant subspace on which to sample.

Other Implementation Aspects

- ▶ Bad approximate solutions (“cheat solutions”)
 - ▶ Neural network training may get stuck in bad approximate solutions.
(e.g. constant, non-increasing, or non-concave V)
 - ▶ We deal with this by **penalizing shape** that corresponds to known bad solutions.
 - ▶ Alternative: train ϕ satisfying $V(a, n, z, g) = \phi(a, n, z, g; \theta)(a - \underline{a})^{1-\gamma}$ instead of V
 - ▶ We also explore complementary ways of addressing this.
(initialization, sampling, network architecture)
- ▶ Boundary conditions
 - ▶ Previous algorithm assumes no boundary condition.
 - ▶ To apply it, we “**soften**” hard constraints in the model.
 - ▶ Alternative: draw sample S^b on boundary and add boundary error $\mathcal{E}^b$ to loss function.
But: hard constraints represent significant difficulty for robustness of our algorithm.

Some Implementation Details

	Finite Population	Discrete State	Projection
Neural Network			
(i) Structure	Feed-forward	Recurrent + embedding	Recurrent + embedding
(ii) Initialization	$W(a, \cdot) = e^{-a}$	random	random
Sampling			
(i) (a, l)	Active sampling $[\underline{a}, \bar{a}] \times \{y_1, y_2\}$	Uniform sampling $[\underline{a}, \bar{a}] \times \{y_1, y_2\}$	Uniform sampling $[\underline{a}, \bar{a}] \times \{y_1, y_2\}$
(ii) $(\hat{\varphi}_i)_{i \leq N}$	Moment sampling: sample r , then random agents positions that give r	Mixed steady-state sampling then ergodic sampling	Sample K and then orthogonal coefficients from ergodic sampling
(iii) z	$U[z_{min}, z_{max}]$	$U[z_{min}, z_{max}]$	$U[z_{min}, z_{max}]$
Loss Function			
(i) Constraints	$\partial_{aa} V(a, \cdot), \partial_{za} V(a, \cdot) < 0$	$\partial_{aa} V(a, \cdot), \partial_{za} V(a, \cdot) < 0$	$\partial_{aa} V(a, \cdot), \partial_{za} V(a, \cdot) < 0$
(ii) Weights	$\kappa^e = 100, \kappa^s = 1$	$\kappa^e = \kappa^s = 1$	$\kappa^e = \kappa^s = 1$

Neural Network Q & A

- ▶ *Q. What are the main differences to discrete time?*
 - ▶ Need to **calculate derivatives rather than expectations** (we use automatic differentiation)
 - ▶ Need to **choose where to sample** (sample more where master equation error is large)
- ▶ *Q. What about slowing down the updating?*
 - ▶ For projection methods, we use “Howard improvement algorithm” to slow down the rate of updating (fix policy rule for some iterations and just update V).
 - ▶ [Duarte, 2018] and [Gopalakrishna, 2021] suggest introducing a “false” time step but so far we have not found this necessary (or found a way to implement at high scale).
 - ▶ We **use shape constraints as a replacement**.
- ▶ *Q. What about imposing symmetry and/or dimension reduction?*
 - ▶ [Han et al., 2021] and [Kahou et al., 2021] suggest feeding the distribution through a preliminary neural network that reduces the dimension and imposes symmetry.
 - ▶ We find we **can solve the problem with and without this approach**.

Table of Contents

Simple Macroeconomic Model (Krusell-Smith (1998))

Solution Algorithm

Testing the Algorithm

Additional Models

Testing the Algorithm

- ▶ We test the model with fixed aggregate productivity [Aiyagari, 1994]

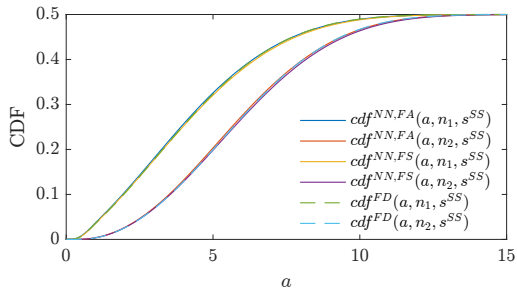
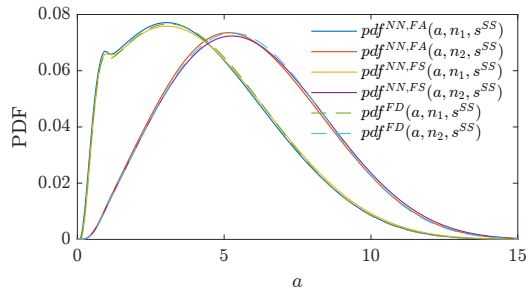
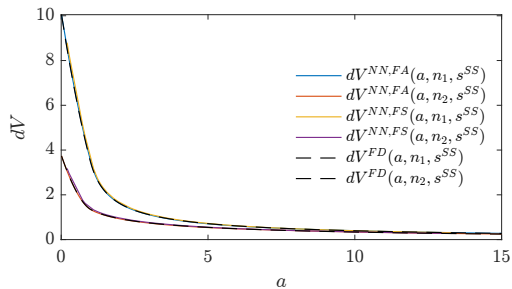
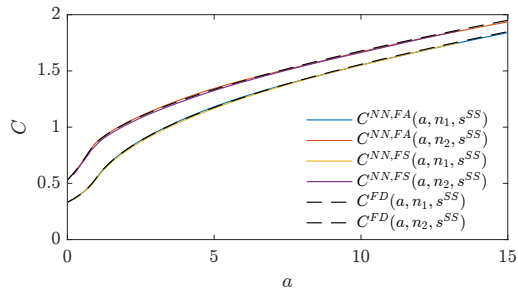
	Master equation loss	MSE(NN, FD)
Finite Agent NN	3.135×10^{-5}	4.758×10^{-5}
Discrete State Space NN	9.303×10^{-6}	6.591×10^{-5}

- ▶ Solve version with stochastic aggregate productivity [Krusell and Smith, 1998]:

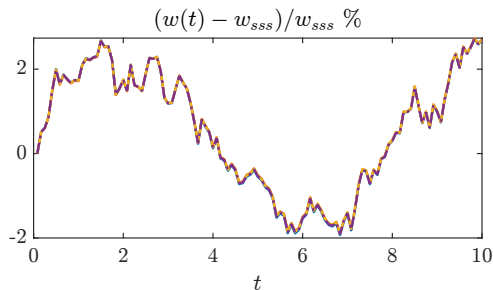
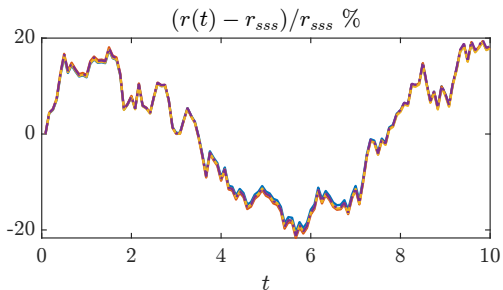
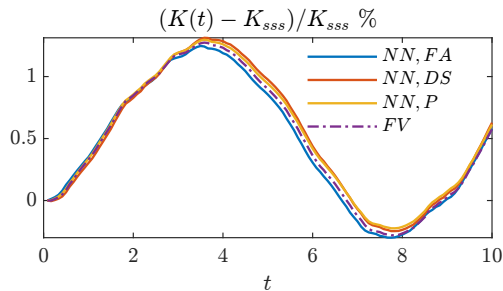
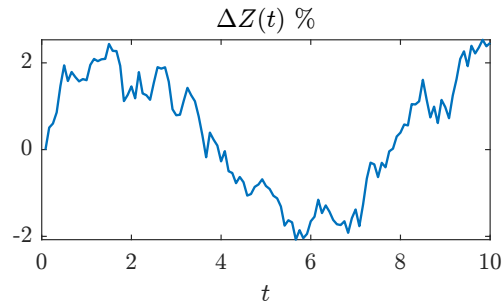
	Master equation training loss
Finite Agent NN	3.037×10^{-5}
Discrete State Space NN	9.639×10^{-5}
Projection NN	8.506×10^{-6}

- ▶ Neural network solutions generate similar output to traditional methods.
- ▶ Example plots (next slide): comparison to [Fernández-Villaverde et al., 2018]
- ▶ Robustness: run each algorithm repeatedly, results are similar across runs

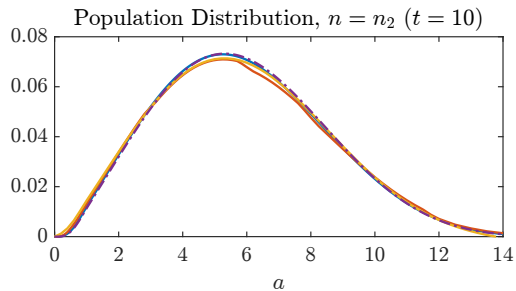
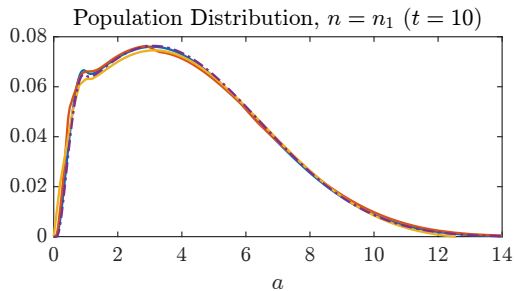
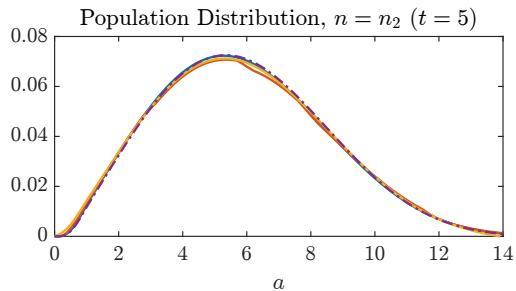
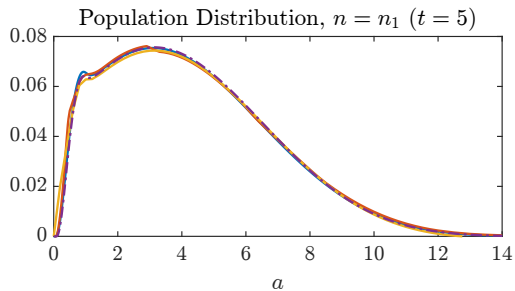
Aiyagari Model: Neural Network and Finite Difference Solution



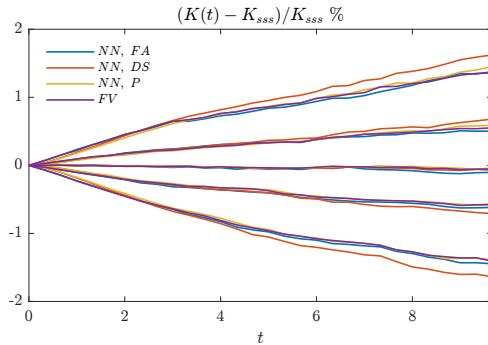
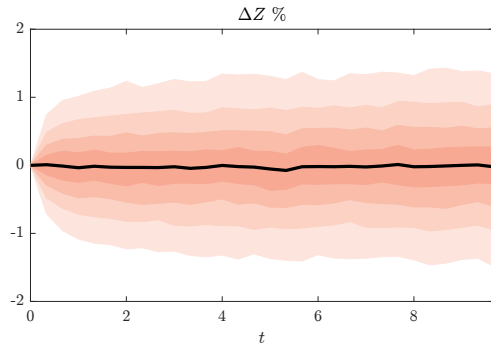
Krusell-Smith: Sample Time Paths



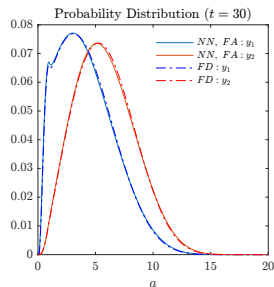
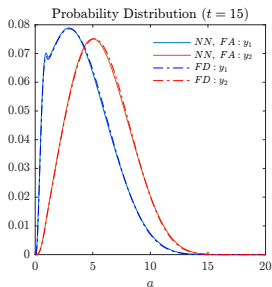
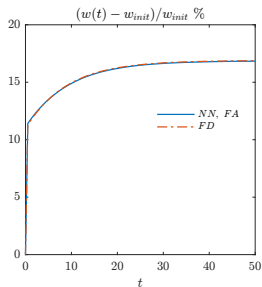
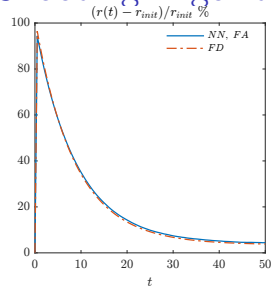
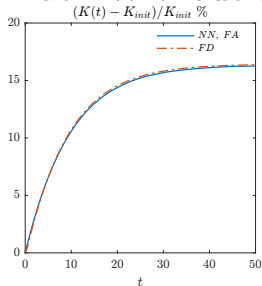
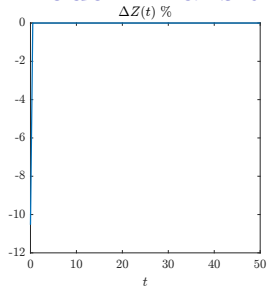
Krusell-Smith: Population Distribution



Krusell-Smith: Fan Chart



Aiyagari Model: Transition Path Without a Shooting Algorithm



Reflections on Different Techniques

- ▶ Finite agent was robust in a number of ways:
 - ▶ The neural network can be trained with the moment sampling procedure
 - ▶ The algorithm only requires a moderate number of agents (approximately 40).
- ▶ Discrete-state method to be difficult to work with:
 - ▶ Having to approximate the derivatives in the KFE is hard.
 - ▶ One challenge is that this requires a fine grid for agent wealth.
 - ▶ Also, the training samples need to come from relatively smooth densities and so ergodic sampling is very important. This made the approach slow and complicated.
- ▶ Projection method brings a different set of trade-offs
 - ▶ Other approaches feed relatively large state spaces into the neural network and then let the neural network work out how to structure the approximate value function.
 - ▶ By contrast, the projection method requires more choices ex-ante.

Table of Contents

Simple Macroeconomic Model (Krusell-Smith (1998))

Solution Algorithm

Testing the Algorithm

Additional Models

Additional Models That Fit This Framework

1. Heterogeneous firm dynamics:

- ▶ In KS-98, there is only one firm.
- ▶ We extend our method to models with many firms that each have non-tradable capital and capital adjustment costs (a version of [Khan and Thomas, 2008])
- ▶ Technical difference is that we need to price firm equity.

2. Dynamic spatial model:

- ▶ In the KS-98 model, there is one wage in the entire economy
- ▶ We extend our method to a spatial model where there are different wages in different locations and agents can move (a version of [Bilal, 2021]).
- ▶ Technical difference is that the wage in location j only depends on the part of the population distribution at location j .

Conclusion and Practical Lessons

- ▶ New algorithm to solve heterogeneous agent models with aggregate shocks.
- ▶ Three methods for approximating population distribution
 - ▶ Which method works best depends on how distribution interacts with agent decision making and complexity of KFE.
 - ▶ For KS model: all methods effective, but finite agent approach is simplest and appears most robust.
- ▶ Implementation details matter! Some practical lessons:
 1. Working out the correct sampling approach is very important.
 2. Neural networks find it easier to work with smooth constraints.
 3. Enforcing shape constraints helps with speed and stability.
 4. Starting with a simple, solvable model is helpful for tuning hyperparameters.

References I



Aiyagari, S. R. (1994).
Uninsured idiosyncratic risk and aggregate saving.
The Quarterly Journal of Economics, 109(3):659–684.



Bilal, A. (2021).
Solving heterogeneous agent models with the master equation.
Technical report, Technical report, University of Chicago.



Duarte, V. (2018).
Machine learning for continuous-time economics.
Available at SSRN 3012602.



Fernández-Villaverde, J., Hurtado, S., and Nuño, G. (2018).
Financial Frictions and the Wealth Distribution.
Working Paper, pages 1–51.



Gopalakrishna, G. (2021).
Aliens and continuous time economies.
Swiss Finance Institute Research Paper, (21-34).



Han, J., Yang, Y., and E, W. (2021).
DeepHAM: A global solution method for heterogeneous agent models with aggregate shocks.
arXiv preprint arXiv:2112.14377.

References II



Kahou, M. E., Fernández-Villaverde, J., Perla, J., and Sood, A. (2021).
Exploiting symmetry in high-dimensional dynamic programming.
Technical report, National Bureau of Economic Research.



Khan, A. and Thomas, J. K. (2008).
Idiosyncratic shocks and the role of nonconvexities in plant and aggregate investment dynamics.
Econometrica, 76(2):395–436.



Krusell, P. and Smith, A. A. (1998).
Income and Wealth Heterogeneity in the Macroeconomy.
Journal of Political Economy, 106(5):867–896.



Sirignano, J. and Spiliopoulos, K. (2018).
Dgm: A deep learning algorithm for solving partial differential equations.
Journal of computational physics, 375:1339–1364.